

Hardware Acceleration of a Simple Ring-LWE Algorithm in Post-Quantum Cryptography

Atit Pokharel

I. BACKGROUND AND INTRODUCTION

A. Background

Numerous internet sources and surveys suggest that over 400 million terabytes of data flow through the internet daily [1]. Cryptography and encryption are the mathematical shields that protect this data from being tapped or even altered by an unauthorized third party. Modern public key systems like RSA and elliptic curve cryptography rely on the presumed hardness of solving numerical problems like prime factorization and discrete logarithms. Classical computers cannot solve these kinds of problems efficiently. To put into perspective, a powerful classical computer will take a trillion years or more to break RSA-2048 using brute force or current best factoring methods. As this seems practically impossible, these encryption security schemes have dominated secure communication for decades [2].

On the other hand, quantum computing is advancing rapidly with the increase in global investments, research breakthroughs and experimental capabilities. In 1994, Peter Shor introduced a quantum algorithm that can efficiently solve both prime factorization and the discrete log problem [3]. This contribution broke the core assumption of hardness behind RSA and ECC. However, these algorithms have not yet been applied in real attacks because today's quantum machines are still significantly noisy, unstable and limited in scale. Yet, recent progress in error correction, qubit scaling and quantum processors' availability suggests that practical and fault-tolerant quantum computers may emerge in a short time [4]. One might think that even if these speculations are accurate, we still have a few years before starting to act on this matter. But the urgency of quantum-safe algorithms comes from an indirect but quite effective attack scheme like "Harvest Now, Decrypt Later" [5]. In this scheme, an unauthorized person can store the encrypted information locally and decrypt it later when the resources are available. Sensitive information of the present, such as government secrets or financial data, is still very valuable in the future and shouldn't fall into the wrong hands.

Recognizing the threat, NIST (National Institute of Standards and Technology) took an initiative to prepare for a cryptographic migration before the quantum era arrives. A multi-year global competition was conducted to evaluate, test, and standardize new quantum-safe algorithms that could replace the vulnerable classical schemes. As a result, NIST finalized a few algorithms, such as CRYSTALS Kyber for key establishment and CRYSTALS Dilithium and Falcon for digital signatures. Notably, several of these selections rely on Learning With Errors (LWE), a lattice-based hardness assumption [6]. In simple terms, LWE asks an attacker to recover a hidden secret from noisy linear equations and it is a task for which no efficient classical or quantum algorithm is currently known.

B. Motivation

To improve efficiency, algorithms like CRYSTAL-Kyber use the Ring-LWE variant. This method operates on polynomial rings rather than matrices. However, this still introduces a heavy

reliance on polynomial multiplication and modular reduction. On general-purpose processors, these operations are quite expensive and typically scale quadratically [7]. This seems to be the computational price the future of computing has to bear in exchange for security. This motivates us to design a hardware accelerator that offloads these computationally expensive polynomial operations to dedicated hardware. With a hardware-software co-design approach, this project tends to bridge the performance gap and demonstrate a functional prototype for a faster execution of a post-quantum cryptographic method.

C. Goals and Objectives

The key objective of this project is to design a functional hardware-software prototype of a Ring-LWE accelerator. The specific goals are as follows:

- **Design a Dedicated Accelerator:** To implement a custom hardware peripheral capable of performing polynomial multiplication. This addresses the primary computational bottleneck of the algorithm.
- **System Integration:** To integrate the hardware accelerator peripheral with the ARM processor on the DE1-SoC platform.
- **Functional Verification:** To validate the correctness of the hardware design by comparing its output against a reference software implementation.
- **Performance Benchmarking:** To profile the system using hardware performance counters and quantify the speedup achieved by the accelerator compared to a software-only implementation.

While the state-of-the-art implementations like Kyber utilize polynomial degrees (or number of coefficients) of $N = 1024$ or 2048 , this prototype targets a smaller parameter set (e.g., $N = 16$). The objective is to demonstrate the core acceleration concepts within manageable hardware complexity constraints.

II. PRIMER ON RING-LWE AND QUANTUM SAFETY

A. The Mathematical Structure

The Ring-Learning With Errors (Ring-LWE) protocol operates within a polynomial ring R_q , formally defined as:

$$R_q = \mathbb{Z}_q[x]/(x^N + 1) \quad (1)$$

In this structure, polynomials have N coefficients, and all arithmetic operations are performed modulo a large prime integer q and modulo the irreducible polynomial $x^N + 1$. This structure allows cryptographic keys to be represented as compact polynomial vectors. For instance, if we take $N = 4$, multiplication with another polynomial of the same order will result in a very high order equation ($N = 4 + 4 = 8$). For larger N , this is infeasible. Thus, the polynomial ring scheme will wrap around in higher orders into lower degree terms using the modulus $x^N + 1$, specifically applying the rule $x^N \equiv -1$ to fold the result back into a fixed size N .

B. Asymmetric Public Key Encryption (PKE)

The project implements an asymmetric PKE scheme involving two parties:

- 1) **Key Generation (Bob):** Bob generates a secret key s and a public key $p = A \cdot s + e$. He broadcasts A and p .

- 2) **Encryption (Alice):** To send a message m , Alice uses Bob's public key p . She samples a random polynomial r and error terms e_1, e_2 to compute two ciphertext components:

$$u = A \cdot r + e_1 \pmod{q} \quad (2)$$

$$v = p \cdot r + e_2 + m \pmod{q} \quad (3)$$

- 3) **Decryption (Bob):** Bob uses his secret key s to recover the message from the ciphertext (u, v) :

$$m' = v - s \cdot u \pmod{q} \quad (4)$$

C. Computational Bottleneck: Polynomial Arithmetic

As evident above, every stage (KeyGen, Encrypt, Decrypt) relies on polynomial multiplication ($A.s, A.r, p.r, s.u$). Unlike integer multiplication, the product $C(x) = A(x) \cdot B(x)$ in the ring R_q expands to a negacyclic convolution, defined as

$$C_k = \sum_{i=0}^{N-1} A_i \cdot B_{(k-i) \pmod{N}} \cdot (-1)^{\lfloor (k-i)/N \rfloor} \pmod{q} \quad (5)$$

This equation essentially means that for any result index k , we calculate the product of every pair A_i and B_j where the indices sum to k or wrap around to $k + N$. If the indices sum directly to k , the product is added. If they wrap around ($i + j \geq N$), the product is subtracted, effectively handling the reduction $x^N \equiv -1$ in a single step. This forces a dependency on every coefficient, resulting in a software time complexity of $\mathcal{O}(N^2)$. So, accelerating this equation is the primary objective of the hardware design.

III. METHODOLOGY

A. Overall System Architecture

The proposed system is implemented on the DE1-SoC platform, which combines a Hard Processor System (HPS) and a programmable FPGA fabric on a single Intel Cyclone V chip. The HPS part consists of a dual-core ARM-Cortex A9 processor responsible for handling software control logic, high-level protocol and C code management. Particularly, all the high-level operations from key generation to encryption-decryption of a sample plaintext are handled in the software, except the polynomial multiplications involved, which is offloaded to the hardware. An example of hardware accelerator integration during the key-generation stage is shown in Fig. 1.

Communication between the HPS and the FPGA is established via high-bandwidth AXI bridges, specifically the Lightweight H2F (Host-to-FPGA) bridge. This interconnect allows the ARM processor to access the hardware accelerator using a memory-mapped interface (Avalon-MM). Thus, the software can write input polynomials to specific register addresses and retrieve the results. To keep track of the computation times, a dedicated 100 MHz hardware Interval Timer is integrated into the system. It is accessible via a memory-mapped interface and provides the precise cycle count required to benchmark the accelerator's performance against the software implementation.

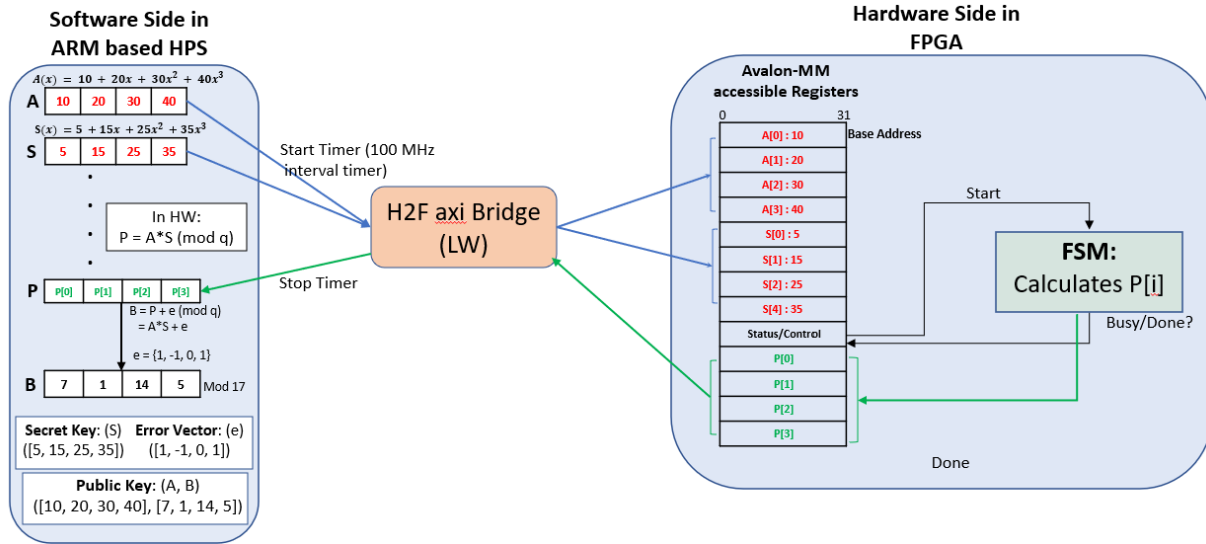


Fig. 1. An overview of HW acceleration in the key generation phase of a Ring-LWE process.

B. Hardware Design (The Accelerator)

The hardware accelerator is designed as a custom IP core that performs polynomial multiplication in the ring $R_q = \mathbb{Z}_q[x]/(x^N + 1)$. The core logic implements negacyclic convolution to handle the polynomial reduction efficiently on-the-fly, eliminating the need for post-multiplication reduction steps.

Finite State Machine (FSM):

The control logic is governed by a Finite State Machine (FSM) with three primary states: IDLE, ACCUM, and DONE.

- **IDLE**: The system waits for a `start_pulse` signal from the software (via the control register). When received, the accumulator is cleared first. Then the internal loop indices are reset before moving to the next state, where the calculation occurs.
- **ACCUM**: This state performs the core arithmetic operations. The behavior depends on the chosen computation method (Sequential or Parallel). However, this part mathematically performs the accumulation of coefficient products. The logic handles the negacyclic property where $x^N = -1$: if the index sum $k = i + j < N$, the product is added; if $k \geq N$, the product is subtracted.
- **DONE**: Once the calculation cycles are done, the FSM pulses a `done` signal. This trigger latches the final accumulation results into the output registers and updates the status register to denote the completion to the software.

Computation Methods:

Two distinct datapath architectures were designed and analyzed:

a. Sequential (Single Multiplier): In this baseline implementation, the hardware mimics the nested loop structure of the software algorithm. It iteratively uses a single hardware multiplier block. The FSM iterates through both inner (j) and outer (i) loops sequentially, resulting in a total latency of N^2 cycles (also FSM overhead).

Although the algorithmic complexity ($O(N^2)$) remains identical to the software approach, speedup

```

assign prod = reg_A[i] * reg_S[j];

ACCUM: begin //in accumulator,
    //negacyclic accumulation (x^16 = -1)
    if (k_idx < 5'd16)
        acc[k_idx] <= acc[k_idx] + prod; //add if k<16
    else
        acc[k_idx - 5'd16] <= acc[k_idx - 5'd16] - prod; //subtract if k>=16
    //increment indices (nested loop logic)
    if (j == 4'd15) begin //if j reaches 15
        j <= 4'd0; //reset j
        if (i != 4'd15) i <= i + 1'b1; //increment i
    end else begin
        j <= j + 1'b1; //increment j
    end
end
end

```

Fig. 2. SystemVerilog code snippet of the ACCUM state, where the sequential logic of software with two nested loops is shown. This resides inside the always_ff block that executes every clock cycle. Essentially, each cycle can have one multiplication operation out of N^2 at a time.

```

ACCUM: begin //in accumulator,
    //so in each cc, one elem is A is used, indexed by i, incremented later
    //outer loop case solved, now inner loop is to be parallelized

    //logic is, if i did 16 inner calc acc[k] += A[i] * S[j] at once, and add them in acc,
    //it wont affect my result, so, i can do this in parallel.
    for (int k = 0; k < 16; k++) begin //so, 16 inner operations at once, it ll use 16 mul
        if (k >= i) begin //same logic, previously, i+j to get k, here, I index k so
            //i can get, k-i to get j, so in P[k] : mul and add/sub of A[i] and S[k-i]
            acc[k] <= acc[k] + (reg_A[i] * reg_S[k - i]);
        end else begin
            acc[k] <= acc[k] - (reg_A[i] * reg_S[16 + k - i]);
        end
    end
    //increment outer loop index
    if (i != 4'd15) begin
        i <= i + 1'b1;
    end
end
end

```

Fig. 3. SystemVerilog code snippet of the ACCUM state, where the inner loop that iterates over S and does the multiplication and accumulation in parallel. Here, each cycle can do N multiplication operations at a time, reducing overall complexity to $O(N)$.

is achieved by eliminating the instruction fetch, decode, and execute overhead of the CPU. Furthermore, the hardware pipeline performs the multiply, index comparison ($k \geq N$), and accumulation (add/subtract) in a single clock cycle. In contrast, the CPU would require multiple instructions for the same operation. The main code snippet adopting this logic is shown in Fig. 2. Here, i is the outer loop index iterating over elements of A , j is the inner loop index iterating over elements of S and $k_idx = i + j$ identifies the target degree in the result polynomial where the current product, $prod = A[i] \cdot S[j]$, must be accumulated with appropriate logic (add or subtract).

b. Parallel (Array of Multipliers): In this implementation, the inner loop is fully unrolled to utilize the FPGA's high-performance parallelism capability. The design instantiates $N = 16$ multiplier blocks operating concurrently as shown in the Fig. 3.

For each iteration of the outer loop (i), the system broadcasts the coefficient $reg_A[i]$ to

all 16 multipliers, each simultaneously multiplying it by a distinct $\text{reg_S}[j]$. This effectively performs the entire inner convolution row in a single step, reducing the total execution time from N^2 iterations to just N iterations. This architecture maximizes throughput at the cost of increased hardware resources.

Register Interface:

For N degree polynomial implementation, the accelerator integrates a memory-mapped register file accessible via the Avalon-MM interface using $3N + 1$ registers. For instance, in the key generation stage, A constitutes N , S constitutes N and the result P needs N , making $3N$ required in total. An additional register is required to have a control and status bit, resulting in a total of $3N + 1$ registers. If we look closely at a $N = 16$ implementation, the registers interface is as follows:

- **Inputs:** Two arrays of 16-bit signed integers, reg_A (Offsets 0–15) and reg_S (Offsets 16–31).
- **Control:** A status/control register at Offset 32. Writing bit 0 triggers the start; reading bit 1 indicates the *busy/done* status.
- **Outputs:** A result array reg_P (Offsets 33–48) storing 32-bit signed integers to accommodate potential overflow before modular reduction.

Hardware Accelerator as a Peripheral and Cost Analysis:

The accelerators were added as external peripherals to the existing DE1-SoC computer system, illustrated in Fig. 4. The accelerator interfaces with the HPS via the Lightweight H2F AXI bridge using an Avalon-MM bus, where global clk and reset_n signals drive the sequential logic. Data transfer is handled by the Avalon-MM interface with signals such as chipselect , write , and writedata for sending polynomial coefficients and control commands. The read , address , and readdata signals allow the processor to get computed results and poll status flags.

Fig. 5 shows the hardware cost of the added peripherals, particularly focusing on the sequential and parallel methods of computation. The sequential one remains lightweight, using only 724 ALMs, 1698 registers and a single DSP block. However, the parallel one trades resource area for performance. It uses 1235 ALMs, 1730 registers and 16 DSP blocks to support the concurrent multiplier array.

C. Software Design

The software is implemented in bare-metal C running on the ARM Cortex-A9 processor. It serves as the master controller for the cryptographic protocol, handling the high-level logic for Key Generation, Encryption, and Decryption.

Driver Interface: The software interacts with the hardware using direct memory access macros (mmio_write and mmio_read) pointing to the accelerator's base address ($0\text{xFF}200000 + \text{offset}$). The driver function $\text{timed_ring_mult16_hw}$ encapsulates the acceleration process through the following sequence:

- Writes the coefficients of the input polynomials A and S to the memory-mapped FPGA registers.
- Writes 0×1 to the control register to trigger the start of the computation.
- Polls the status register in a `while` loop until the "done" bit is asserted. This ensures that the hardware has completed the computation.

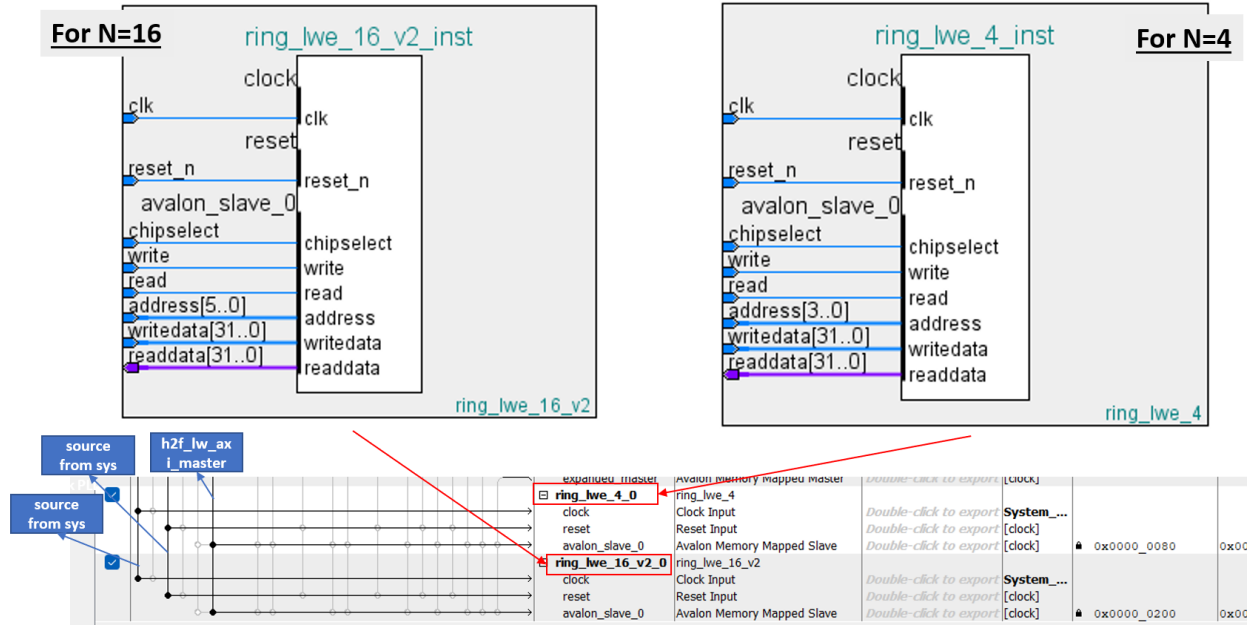


Fig. 4. Signals and Interfaces of the peripherals (accelerators) integrated in the existing DE1-SoC computer system.

Compilation Hierarchy Node	ALMs needed (A+B+C)	ALMs used in final placement	Estimate of ALMs recoverable by dense pack	Estimate of ALMs unavailable	Combinational ALUTs	Dedicated Logic Registers	I/O Registers	Memo	M10Ks	DSP Blocks
IDE1_SoC_Computer	21259.5 (1.9)	23930.5 (2.4)	2917.5 (0.5)	246.5 (0.0)	34648 (6)	28240 (0)	432 (432)	22_34_321	28	
Computer_System:The_System	21044.0 (0.0)	23688.6 (0.0)	2891.0 (0.0)	246.5 (0.0)	34300 (0)	28036 (0)	0 (0)	22_34_321	28	
Computer_System_min_interconnect_0:mm_interconnect_0	7393.6 (0.0)	7804.5 (0.0)	511.4 (0.0)	100.5 (0.0)	12194 (0)	6980 (0)	0 (0)	128_1	0	
ring_lwe_multi6_mmio_parallelring_lwe_16_v2_0	724.2 (724.2)	1062.7 (1062.7)	341.5 (341.5)	3.0 (0.0)	815 (815)	1698 (1698)	0 (0)	0_0	1	
ring_lwe_multi6_mmio_parallelring_lwe_16_parallel_0	1234.6 (1234.6)	1415.6 (1415.6)	168.4 (168.4)	7.7 (7.7)	1555 (1555)	1730 (1730)	0 (0)	0_0	16	
ring_lwe_multi4_mmio_parallelring_lwe_4_0	208.7 (133.0)	255.6 (164.1)	48.1 (32.3)	1.2 (1.2)	352 (238)	423 (270)	0 (0)	0_0	1	

Fig. 5. Cost analysis of the added peripherals.

- It reads the resulting polynomial P from the output registers for further processing.

To validate speedup, the software uses the system's 100 MHz hardware interval timer. It captures cycle counts before and after the hardware and software function calls to calculate precise execution times.

Cryptographic Protocol Implementation: The software integrates the hardware driver into the full Ring-LWE scheme to accelerate the critical multiplication steps:

- **Key Generation:** The software samples the public vector A and secret s . It offloads the calculation of the product $A \cdot s$ to the hardware accelerator and subsequently adds the error term e in software, followed by modulo reduction with modulus Q to produce the public key b . The
- **Encryption:** To encrypt a message m , the software must compute the vectors $u = A \cdot r + e_1 \pmod{Q}$ and $v = b \cdot r + e_2 + m \pmod{Q}$. This operation requires two distinct calls to the hardware accelerator to compute the intermediate products $A \cdot r$ and $b \cdot r$.
- **Decryption:** The software recovers the message by computing $v - s \cdot u$. It utilizes the hardware to multiply the ciphertext component u by the secret key s , subtracts this result from v , and performs threshold rounding to retrieve the original message bits.

IV. EXPERIMENTS AND RESULTS

A. Case I: $N=4$ (Proof of Concept)

This initial experiment serves as a proof of concept to validate the hardware acceleration of polynomial multiplication. Before moving further with the higher complexity of the cryptographic scheme, my focus was mainly on the Key Generation and Encryption steps to verify the correctness of the hardware acceleration on a small scale ($N = 4$) before scaling up.

In this setup, the hardware accelerator achieved a cycle count of 87 for Key Generation compared to 475 in software. During encryption, the hardware required 174 cycles vs. 1410 in software. These results confirmed the functional correctness of the datapath and demonstrated the potential for significant computation time reduction.

Note: All the clock cycle values discussed in this section are from the 100 MHz interval timer. Thus, each clock cycle corresponds to 10 ns in wall clock time.

B. Case II: $N=16$

For the primary experimental case ($N = 16$), we evaluated two distinct hardware architectures: a simple Sequential design and an optimized Parallel design.

1. Sequential Method:

```
Key Gen:
int16_t A[N] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32};
int16_t S[N] = {1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1};
int16_t e[N] = {-1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1};

Encryption/Decryption:
//test data, random
int16_t msg[N] = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
int16_t r[N] = {1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0};
int16_t e1[N] = {-1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1};
int16_t e2[N] = {0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0};
```

Fig. 6. An example of the test values used ($A, S, e, e1, e2, msg$ and r).

This implementation is based on a single-multiplier design discussed in the methodology. The test vectors ($A, S, e1, e2, r$) used for are shown in Fig. 6. The performance results demonstrate a significant speedup, particularly as the computational load increases $N=4$ to 16. The specific clock cycle measurements for Key Generation, Encryption, and Decryption are detailed in Table I. For a comprehensive analysis, the system is evaluated using two distinct methods of profiling, one that includes communication time from HPS to FPGA, along with the computation. Another method profiles clock cycles required solely for the computation.

As shown in Table I, the sequential hardware implementation drastically reduces the cycle count by eliminating the instruction fetch and decode overhead associated with the processor. In terms of raw computation, the accelerator gained massive efficiency with speedups ranging from 40.3x to 43.5x across all the operations. When accounting for communication overhead via AXI bridge, the hardware system gained a total speedup of $(7.4 \pm 1.5)x$. This result already shows the potential of hardware being able to speed up the process extensively, even without true parallelization. With parallelization incorporated, the performance gain can be scaled massively,

TABLE I
PERFORMANCE RESULTS FOR N=16 (SEQUENTIAL ARCHITECTURE): COMPUTATION VS. TOTAL LATENCY

Operation	SW Cycles	Hardware Cycles		Speedup	
		Compute Only	Total (w/ Comm)	Compute	Total
Key Generation	11,380	282	1,714	40.3x	6.7x
Encryption	22,753	542	3,034	41.9x	7.4x
Decryption	11,406	262	1,603	43.5x	7.1x

which is discussed in the next section.

2. *Parallel Method*: To further optimize performance, a parallel architecture was implemented where 16 multiplier units operate concurrently. Here, the inner loop is effectively unrolled during synthesis. The performance metrics for this architecture are summarized in Table II.

TABLE II
PERFORMANCE RESULTS FOR N=16 (PARALLEL ARCHITECTURE): COMPUTATION VS. TOTAL LATENCY

Operation	SW Cycles	Hardware Cycles		Speedup	
		Compute Only	Total (w/ Comm)	Compute	Total
Key Generation	11,468	58	1,400	195.9x	8.1x
Encryption	22,892	116	2,725	196.4x	8.4x
Decryption	11,430	58	1,303	196.4x	8.7x

The parallel design achieves even greater speedups by performing the entire inner product calculation in a single step. When considering the computation time only, the massive speedup of 195x from hardware is achieved. However, when we consider the communication overhead too, the speedup falls to an average of 8.5x. This is still an improvement from the sequential method, but it also reveals an important fact. In this particular experimental setup (DE1-SoC with Cyclone V), the communication overhead backfires on the computation speedup achieved by the hardware.

V. CONCLUSION

This project successfully designed and implemented a hardware-software co-designed prototype for accelerating the Ring-LWE algorithm in the DE1-SoC platform. By targeting the computationally expensive polynomial multiplication operation that scales quadratically on general-purpose processors, the demonstrated system effectively showcased a method to address the performance bottleneck in the modern PQC schemes. Experimental results validated the functional correctness of the computational logic and demonstrated significant performance gains. A critical insight observed from the performance profiling is the dominance of communication overhead in these particular experimental settings. While the hardware accelerator has the potential to accelerate the polynomial multiplication by an order of hundreds, the total speedup, including communication, struggled in a much lesser value of tens. This pointed out the HPS-to-FPGA bridge as a limiting factor. The future works can address the communication overhead and scale the demonstrated prototype to a standardized PQC algorithm that involves much higher degree polynomials ($N = 1024, 2048$).

REFERENCES

- [1] F. Duarte, “Amount of data created daily (2025),” <https://explodingtopics.com/blog/data-generated-per-day>, April 2025, exploding Topics. Accessed: 2025-12-08.
- [2] QuintessenceLabs, “Breaking rsa encryption - an update on the state-of-the-art,” <https://www.quintessencelabs.com/blog/breaking-rsa-encryption-update-state-art>, June 2019, accessed: 2025-12-08.
- [3] P. W. Shor, “Algorithms for quantum computation: discrete logarithms and factoring,” in *Proceedings 35th annual symposium on foundations of computer science*. Ieee, 1994, pp. 124–134.
- [4] Google Quantum AI, “Suppressing quantum errors by scaling a surface code logical qubit,” *Nature*, vol. 614, no. 7949, pp. 676–681, 2023. [Online]. Available: <https://www.nature.com/articles/s41586-022-05434-1>
- [5] H. Singh, “Managing the quantum cybersecurity threat: Harvest now, decrypt later,” in *Quantum Computing*. CRC Press, 2024, pp. 142–158.
- [6] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, “Crystals-kyber: a cca-secure module-lattice-based kem,” in *2018 IEEE European symposium on security and privacy (EuroS&P)*. IEEE, 2018, pp. 353–367.
- [7] R. Salarifard and H. Soleimany, “An efficient hardware accelerator for ntt-based polynomial multiplication using fpga,” *Journal of Cryptographic Engineering*, vol. 14, no. 2, pp. 415–426, 2024.

APPENDIX

SystemVerilog Code for the Parallel Version with 16 Multipliers

```

1  module ringlwe_mult16_mmio_parallel (
2      input  logic clk, reset_n, chipselect, write, read,
3      input  logic [5:0] address,
4      input  logic [31:0] writedata,
5      output logic [31:0] readdata
6  );
7
8      logic status_done, start_pulse, fsm_busy, fsm_done_pulse;
9
10     //lets have some registers, A, S and P(32b)
11     logic signed [15:0] reg_A [0:15];
12     logic signed [15:0] reg_S [0:15];
13     logic signed [31:0] reg_P [0:15]; // Result registers
14
15     //custom struct for fsm
16     typedef enum logic [1:0] {IDLE = 2'b00, ACCUM = 2'b01, DONE = 2'b10}
17     ⇨ my_state_type;
18     my_state_type cur_state, next_state;
19
20     //index for outer loop
21     logic [3:0] i; //can go up to 15 (16 elems), outer loop index
22     //accumulator for the result
23     logic signed [31:0] acc [0:15];
24     assign fsm_busy = (cur_state != IDLE); //busy if not idle
25
26     //next state logic for the fsm
27     always_comb begin
28         next_state = cur_state;
29         case(cur_state)
30             IDLE: if (start_pulse) next_state = ACCUM;
31             //so, when the index reaches 16, one round is done, go to done
32             ACCUM: if (i == 4'd15) next_state = DONE;
33             DONE: next_state = IDLE;
34             default: next_state = IDLE;
35         endcase
36     end

```

```

36
37 //main sequential logic
38 always_ff @(posedge clk or negedge reset_n) begin
39     if (!reset_n) begin //reset the registers
40         for (int k = 0; k < 16; k++) begin //reset all registers
41             reg_A[k] <= 16'sd0;
42             reg_S[k] <= 16'sd0;
43             reg_P[k] <= 32'sd0;
44         end
45         status_done <= 1'b0; //ctrl and stat = 0
46         start_pulse <= 1'b0;
47         cur_state <= IDLE; //index reset. cur_state to idle
48         i <= 4'd0;
49         acc <= 32'sd0;
50         fsm_done_pulse <= 1'b0;
51
52     end else begin //if not reset,
53         start_pulse <= 1'b0; //lets make sure, default point is 0
54         fsm_done_pulse <= 1'b0;
55         cur_state <= next_state; //cur_state to next state
56
57         case (cur_state)
58             IDLE: begin //in idle, if we get start pulse, first clear acc and
59                 ↪ reset the index, and it begins
60                 if (start_pulse) begin
61                     acc <= 32'sd0;
62                     i <= 4'd0;
63                 end
64             end
65             ACCUM: begin //in accumulator,
66                 //so in each cc, one elem is A is used, indexed by i,
67                 ↪ incremented later
68                 //outer loop case solved, now inner loop is to be parallelized
69
70                 //logic is, if i did 16 inner calc acc[k] += A[i] * S[j] at
71                 ↪ once, and add them in acc,
72                 //it wont affect my result, so, i can do this in parallel.
73                 for (int k = 0; k < 16; k++) begin //so, 16 inner operations
74                     ↪ at once, it ll use 16 mul
75                     if (k >= i) begin //same logic, previously, I used to do
76                         ↪ i+j to get k, here, I index k so
77                         //i can get, k-i to get j, so what value to put in P[k] :
78                         ↪ mul and add/sub of A[i] and S[k-i]
79                         acc[k] <= acc[k] + (reg_A[i] * reg_S[k - i]);
80                     end else begin
81                         acc[k] <= acc[k] - (reg_A[i] * reg_S[16 + k - i]);
82                     end
83                 end
84             end
85
86             //increment outer loop index
87             if (i != 4'd15) begin
88                 i <= i + 1'b1;
89             end
90         end
91     end
92 end

```

```

186         DONE: begin
187             fsm_done_pulse <= 1'b1; //done pulse to 1
188             reg_P <= acc; //acc to P
189             status_done <= 1'b1; //done status to 1
190         end
191     endcase
192
193     //write logic
194     if (chipselct && write) begin
195         if (address == 6'd32) begin
196             if (writedata[0]) begin //start bit
197                 status_done <= 1'b0; //clear done status
198                 start_pulse <= 1'b1; //trigger fsm start
199             end
200         end
201         else if (!fsm_busy) begin //only allow writing inputs if fsm is
202             ↪ idle
203             if (address < 6'd16) begin
204                 reg_A[address] <= writedata[15:0];
205             end else if (address >= 6'd16 && address < 6'd32) begin
206                 // Address 16-31: reg_S (Offset by 16)
207                 reg_S[address - 6'd16] <= writedata[15:0];
208             end
209         end
210     end
211 end
212
213 //read logic, same logic as before, just read the registers
214 always_comb begin
215     readdata = 32'd0;
216     if (chipselct && read) begin
217         if (address < 6'd16) begin
218             readdata = {{16{reg_A[address][15]}}, reg_A[address]};
219         end
220         else if (address >= 6'd16 && address < 6'd32) begin
221             readdata = {{16{reg_S[address - 6'd16][15]}}, reg_S[address -
222             ↪ 6'd16]};
223         end
224         else if (address == 6'd32) begin
225             readdata = {30'd0, status_done, fsm_busy};
226         end
227         else if (address >= 6'd33 && address < 6'd49) begin
228             //read result p, offset by 33
229             readdata = reg_P[address - 6'd33];
230         end
231     end
232 end
233 endmodule

```

SystemVerilog Code for the Sequential Version with 1 Multiplier

```

1 module ringlwe_mult16_mmio_flat (
2     input logic clk, reset_n, chipselct, write, read,
3     input logic [5:0] address,

```

```

4     input  logic [31:0] writedata,
5     output logic [31:0] readdata
6 );
7
8     logic status_done, start_pulse, fsm_busy, fsm_done_pulse;
9
10    //registers, A, S and P(32b)
11    logic signed [15:0] reg_A [0:15];
12    logic signed [15:0] reg_S [0:15];
13    logic signed [31:0] reg_P [0:15]; // Result registers
14
15    typedef enum logic [1:0] {IDLE = 2'b00, ACCUM = 2'b01, DONE = 2'b10}
16    ⇐ my_state_type;
17    my_state_type cur_state, next_state;
18
19    //indices for nested loop
20    logic [3:0] i, j; //can go up to 15 (16 elems)
21    logic [4:0] k_idx; //max sum is 15+15=30, needs 5 bits
22
23    //accumulator for the result
24    logic signed [31:0] acc [0:15];
25    logic signed [31:0] prod;
26
27    assign prod = reg_A[i] * reg_S[j]; //product of current indexed elements
28    assign k_idx = i + j; //sum of indices
29    assign fsm_busy = (cur_state != IDLE); //busy if not idle
30
31    //next state logic for the fsm
32    always_comb begin
33        next_state = cur_state;
34        case (cur_state)
35            IDLE: if (start_pulse) next_state = ACCUM;
36            //finish when both i and j reach 15
37            ACCUM: if (i == 4'd15 && j == 4'd15) next_state = DONE;
38            DONE: next_state = IDLE;
39            default: next_state = IDLE;
40        endcase
41    end
42
43    //main sequential logic
44    always_ff @(posedge clk or negedge reset_n) begin
45        if (!reset_n) begin //reset the registers
46            for (int k = 0; k < 16; k++) begin //reset all registers
47                reg_A[k] <= 16'sd0;
48                reg_S[k] <= 16'sd0;
49                reg_P[k] <= 32'sd0;
50            end
51            status_done <= 1'b0; //ctrl and stat - 0
52            start_pulse <= 1'b0;
53            cur_state <= IDLE; //cur_state to idle
54            i <= 4'd0;
55            j <= 4'd0;
56            acc <= '{default: 32'sd0};
57            fsm_done_pulse <= 1'b0;
58
59            end else begin //if not reset,

```

```

59     start_pulse <= 1'b0; //lets make sure, default point is 0
60     fsm_done_pulse <= 1'b0;
61     cur_state <= next_state; //cur_state to next state
62
63     case (cur_state)
64         IDLE: begin //in idle, if we get start pulse, first clear acc and
        ↪ reset the indices, and it begins
65             if (start_pulse) begin
66                 acc <= '{default: 32'sd0};
67                 i <= 4'd0;
68                 j <= 4'd0;
69             end
70         end
71
72         ACCUM: begin //in accumulator,
73             //negacyclic accumulation (x^16 = -1)
74             if (k_idx < 5'd16)
75                 acc[k_idx] <= acc[k_idx] + prod; //add if k<16
76             else
77                 acc[k_idx - 5'd16] <= acc[k_idx - 5'd16] - prod;
        ↪ //subtract if k>=16, wrapping around
78
79             //increment indices (nested loop logic)
80             if (j == 4'd15) begin //if j reaches 15
81                 j <= 4'd0; //reset j
82                 if (i != 4'd15) i <= i + 1'b1; //increment i
83             end else begin
84                 j <= j + 1'b1; //increment j
85             end
86         end
87
88         DONE: begin
89             fsm_done_pulse <= 1'b1; //done pulse to 1
90             reg_P <= acc; //acc to P
91             status_done <= 1'b1; //done status to 1
92         end
93     endcase
94
95     //write logic
96     if (chipselct && write) begin
97         if (address == 6'd32) begin
98             if (writedata[0]) begin //start bit
99                 status_done <= 1'b0; //clear done status
100                 start_pulse <= 1'b1; //trigger fsm start
101             end
102         end
103         else if (!fsm_busy) begin //only allow writing inputs if fsm is
        ↪ idle
104             if (address < 6'd16) begin
105                 reg_A[address] <= writedata[15:0];
106             end else if (address >= 6'd16 && address < 6'd32) begin
107                 // Address 16-31: reg_S (Offset by 16)
108                 reg_S[address - 6'd16] <= writedata[15:0];
109             end
110         end
111     end

```

```

112     end
113 end
114
115 //read logic, same logic as before, just read the registers
116 always_comb begin
117     readdata = 32'd0;
118     if (chipselct && read) begin
119         if (address < 6'd16) begin
120             readdata = {{16{reg_A[address][15]}}, reg_A[address]};
121         end
122         else if (address >= 6'd16 && address < 6'd32) begin
123             readdata = {{16{reg_S[address - 6'd16][15]}}, reg_S[address -
124                 ↪ 6'd16]};
125         end
126         else if (address == 6'd32) begin
127             readdata = {30'd0, status_done, fsm_busy};
128         end
129         else if (address >= 6'd33 && address < 6'd49) begin
130             //read result p, offset by 33
131             readdata = reg_P[address - 6'd33];
132         end
133     end
134 end
135 endmodule

```

Final C code:

```

1  #include <stdio.h>
2  #include <stdint.h>
3  #include <string.h>
4  #include <stdlib.h>
5  //addresses
6  #define AVALON_BRIDGE_BASE 0xFF200000
7  #define RING_LWE_OFFSET 0x00000200
8  //so the component base becomes
9  #define RING_LWE_BASE (AVALON_BRIDGE_BASE + RING_LWE_OFFSET)
10 #define TIMER_MM 0xFF202000 //timer base address, 100 Mhz
11
12 //registers offsets, so, when r/w, I can do like: ring_lwe_base +
13 ↪ 4*(reg_a_base_idx + some offset)
14 #define REG_A_BASE_IDX 0
15 #define REG_S_BASE_IDX 16
16 #define REG_CTRL_IDX 32
17 #define REG_P_BASE_IDX 33
18 #define N 16 //n is 16
19 #define Q 257 //modulus=257, modulus 17 became too small for the error I used
20
21 //to read and write, simple inline functions
22 static inline void mmio_write(uint32_t addr, uint32_t data) { //inp args: address
23     ↪ and data
24     volatile uint32_t *p = (uint32_t *)addr;
25     *p = data; //put data in that address
26 }
27 static inline uint32_t mmio_read(uint32_t addr) { //similarly read from the
28     ↪ address

```

```

26     volatile uint32_t *p = (uint32_t *)addr;
27     return *p; //return data, return type uint32
28 }
29
30 //now function for hardware
31 uint32_t timed_ring_mult16_hw(const int16_t A[N], const int16_t S[N], int32_t
↪ P[N]){ //making it general so, I can reuse it for keygen and enc both
32     volatile uint32_t *ptr_to_timer = (uint32_t *)TIMER_MM; //timer address
33     uint32_t last_count, total_cycles; //vars for timer
34     int i; //had to add this declaration, was getting compilation errors
35     *(ptr_to_timer + 3) = 0xFFFF; //max value
36     *(ptr_to_timer + 2) = 0xFFFF;
37     *(ptr_to_timer + 1) = 0x4; // start
38
39     //Putting A in reg A and S in reg S
40     for(i = 0; i < N; i++) { //so, iterate over 16 values and put one by one
41         mmio_write(RING_LWE_BASE + 4*(REG_A_BASE_IDX + i), (uint16_t)A[i]); //16
↪ bit values
42     }
43     for(i = 0; i < N; i++) {
44         mmio_write(RING_LWE_BASE + 4*(REG_S_BASE_IDX + i), (uint16_t)S[i]);
45     }
46     //timer setup
47
48     mmio_write(RING_LWE_BASE + 4 * REG_CTRL_IDX, 0x1); //start the accelerator
49
50     //to check the calc is done or not, polling the stat register
51     volatile uint32_t *status_reg = (uint32_t *) (RING_LWE_BASE + 4 *
↪ REG_CTRL_IDX); //address
52     while (!(*status_reg & 0x2)) { //can see 10 if calc is done
53         //if this branch is taken, fsm is busy, wait
54     }
55     //out of the loop means, calc done
56     *(ptr_to_timer + 1) = 0x8; //stop timer
57
58     //now, reading the result
59     for(i = 0; i < N; i++) {
60         P[i] = (int32_t)mmio_read(RING_LWE_BASE + 4 * (REG_P_BASE_IDX + i));
61     }
62     //time calculation as always
63     *(ptr_to_timer + 4) = 1;
64     last_count = (*(ptr_to_timer + 5) << 16) | *(ptr_to_timer + 4); //shift and
↪ concat two 16 bit values
65     total_cycles = 0xFFFFFFFF - last_count; //diff is the total cycles
66     return total_cycles;
67 } //so, result is returned via mmio read, and total cycles via software in this
↪ function
68
69 //sw based implementation
70 uint32_t timed_ring_mult16_sw(const int16_t A[N], const int16_t S[N], int32_t
↪ P[N]) { //same args
71     volatile uint32_t *ptr_to_timer = (uint32_t *)TIMER_MM; //same timer
72     uint32_t last_count, total_cycles;
73     int i, j, k; //indices
74     int32_t prod; //product, a slack var
75
76     *(ptr_to_timer + 3) = 0xFFFF; //max val

```

```

76     *(ptr_to_timer + 2) = 0xFFFF;
77     *(ptr_to_timer + 1) = 0x4; //start timer
78
79     //init result to 0
80     for (i = 0; i < N; i++) P[i] = 0;
81     //so outer loop, iterate over values of A
82     for (i = 0; i < N; i++) {
83         for (j = 0; j < N; j++) { //inner loop, iterate over values of S
84             k = i + j; //sum of indices is k, used for condition check
85             prod = (int32_t)A[i] * (int32_t)S[j]; //product is required, anyways,
            ↪ so compute it
86             //now based on conditions, add or sub the product to accumulated
            ↪ result
87             if (k < N) { //if k is less than N, add the product to P[k]
88                 P[k] += prod;
89             } else { //if k is greater than or equal to N, sub the product from
            ↪ P[k - N]
90                 P[k - N] -= prod; // Wrap around with negation
91             }
92         }
93     } //same for all values of A
94
95     *(ptr_to_timer + 1) = 0x8; //stop timer
96     *(ptr_to_timer + 4) = 1; //latch
97     last_count = (*(ptr_to_timer + 5) << 16) | *(ptr_to_timer + 4); //shift and
            ↪ concat two 16 bit values
98     total_cycles = 0xFFFFFFFF - last_count; //diff is the total cycles
99
100     return total_cycles; //return total cycles
101 }
102
103 //encryption - args: public key a, public key p, message m, random r, error e1,
            ↪ error e2, output u, output v
104 uint32_t timed_ring_encrypt_hw(const int16_t pk_a[N], const int16_t pk_p[N], const
            ↪ int16_t m[N], const int16_t r[N], const int16_t e1[N], const int16_t e2[N],
105                               int32_t u[N], int32_t v[N]) {
106     int32_t ar[N];
107     int32_t pr[N];
108     int i;
109     int32_t m_scaled; //scaled message
110     uint32_t cycles_ar, cycles_pr; //ar: for u, pr: for v
111
112     //u = a*r + e1 (Hardware Mult) and mod Q
113     cycles_ar = timed_ring_mult16_hw(pk_a, r, ar); //use accelerator
114     for (i = 0; i < N; i++) {
115         u[i] = ar[i] + e1[i];
116         u[i] = ((u[i] % Q) + Q) % Q; //apply modulo Q
117     } //v = p*r + e2 + m_scaled (Hardware Mult) and mod Q, scale m by Q/2
118     cycles_pr = timed_ring_mult16_hw(pk_p, r, pr); //use accelerator
119     for (i = 0; i < N; i++) {
120         m_scaled = m[i] * (Q/2); //scale message by Q/2 = 8
121         v[i] = pr[i] + e2[i] + m_scaled;
122         v[i] = ((v[i] % Q) + Q) % Q;
123     }
124     return cycles_ar + cycles_pr; //total cycles
125 }
126 //same for SW, same args

```

```

126 uint32_t timed_ring_encrypt_sw(const int16_t pk_a[N], const int16_t pk_p[N], const
    ↪ int16_t m[N], const int16_t r[N], const int16_t e1[N], const int16_t e2[N],
127                               int32_t u[N], int32_t v[N]) {
128     int32_t ar[N];
129     int32_t pr[N];
130     int i;
131     int32_t m_scaled; //scaled message
132     uint32_t cycles_ar, cycles_pr;
133     // u = a*r + e1 (Software Mult) and mod Q
134     cycles_ar = timed_ring_mult16_sw(pk_a, r, ar); //use software
135     for (i = 0; i < N; i++) {
136         u[i] = ar[i] + e1[i];
137         u[i] = ((u[i] % Q) + Q) % Q;
138     // v = p*r + e2 + m_scaled (Software Mult) and mod Q, scale m by Q/2
139     cycles_pr = timed_ring_mult16_sw(pk_p, r, pr);
140     for (i = 0; i < N; i++) {
141         m_scaled = m[i] * (Q/2); //scale message by Q/2 = 8
142         v[i] = pr[i] + e2[i] + m_scaled;
143         v[i] = ((v[i] % Q) + Q) % Q;
144     return cycles_ar + cycles_pr;
145 }
146
147 //decryption - args: secret key s, ciphertext u, ciphertext v, output decrypted
    ↪ message m
148 uint32_t timed_ring_decrypt_hw(const int16_t s[N], const int32_t u[N], const
    ↪ int32_t v[N], int32_t m_dec[N]) {
149     int32_t su[N]; //s*u
150     int i;
151     int16_t u16[N]; //need to convert u to int16 for hw function
152     int32_t noisy_m; //noisy message before rounding
153     uint32_t cycles;
154
155     //convert u to int16
156     for(i=0; i<N; i++) u16[i] = (int16_t)u[i];
157
158     //compute s*u using hardware
159     cycles = timed_ring_mult16_hw(s, u16, su);
160
161     //m' = v - s*u (mod Q), then round to recover message bit
162     for(i=0; i<N; i++) {
163         noisy_m = v[i] - su[i];
164         noisy_m = ((noisy_m % Q) + Q) % Q; //apply modulo Q
165         //round: if noisy_m is close to 0 or Q, output 0; if close to Q/2, output
            ↪ 1
166         m_dec[i] = ((noisy_m + Q/4) / (Q/2)) % 2; //rounding to nearest bit
167     }
168     return cycles; //return cycles
169 }
170
171 //decryption SW - same args
172 uint32_t timed_ring_decrypt_sw(const int16_t s[N], const int32_t u[N], const
    ↪ int32_t v[N], int32_t m_dec[N]) {
173     int32_t su[N]; //s*u
174     int i;
175     int16_t u16[N]; //need to convert u to int16 for sw function
176     int32_t noisy_m; //noisy message before rounding

```

```

177     uint32_t cycles;
178
179     //convert u to int16
180     for(i=0; i<N; i++) u16[i] = (int16_t)u[i];
181
182     //compute s*u using software
183     cycles = timed_ring_mult16_sw(s, u16, su);
184
185     //m' = v - s*u (mod Q), then round to recover message bit
186     for(i=0; i<N; i++) {
187         noisy_m = v[i] - su[i];
188         noisy_m = ((noisy_m % Q) + Q) % Q; //apply modulo Q
189         //round: if noisy_m is close to 0 or Q, output 0; if close to Q/2, output
190         ↪ 1
191         m_dec[i] = ((noisy_m + Q/4) / (Q/2)) % 2; //rounding to nearest bit
192     }
193     return cycles; //return cycles
194 }
195
196 //print function for result arrays, needed this, because a lot of printing is
197 ↪ required
198 void print_array(const char *name, const int32_t *p, int n) { //so it is like:
199 ↪ print array name, iterate ove values and print 'em
200     int i;
201     printf("%s = [", name); //name of the array
202     for (i = 0; i < n; i++) {
203         printf("%ld", p[i]);
204         if (i < n - 1) printf(", "); //comma and space between values
205     }
206     printf("]\n"); //line change at end
207
208 //main
209 int main(void) {
210     int i;
211     int16_t A[N] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32};
212     int16_t S[N] = {1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1};
213     int16_t e[N] = {-1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1};
214
215     int32_t P_hw[N]; //result array for hardware
216     int32_t P_sw[N]; //result array for software
217     uint32_t total_cycles_keygen_hw, total_cycles_keygen_sw;
218     uint32_t total_cycles_encrypt_hw, total_cycles_encrypt_sw; //total cycles for
219     ↪ hardware and software
220     uint32_t total_cycles_decrypt_hw, total_cycles_decrypt_sw; //total cycles for
221     ↪ decryption
222
223     //key generation part
224     //prnt the generated values first
225     printf("Key Generation:\n");
226     printf("A = ["); for(i=0; i<N; i++) printf("%d%s", A[i], (i<N-1)?", ":"");
227     ↪ printf("]\n");
228     printf("S = ["); for(i=0; i<N; i++) printf("%d%s", S[i], (i<N-1)?", ":"");
229     ↪ printf("]\n");
230     printf("Error e: = ["); for(i=0; i<N; i++) printf("%d%s", e[i], (i<N-1)?",
231     ↪ ":""); printf("]\n");
232     printf("Calculating in Hardware (a*s + e)...\n");

```

```

225 //now calculate the result using hardware acc
226 total_cycles_keygen_hw = timed_ring_mult16_hw(A, S, P_hw);
227 for(i=0; i<N; i++) {
228     P_hw[i] += e[i]; //add error
229     P_hw[i] = ((P_hw[i] % Q) + Q) % Q; //apply modulo Q
230 } //now my public key is ready
231 printf("Done in HW.\n\n");
232
233 printf("Calculating in Software (a*s + e)...\n"); //same for software
234 total_cycles_keygen_sw = timed_ring_mult16_sw(A, S, P_sw);
235 for(i=0; i<N; i++) {
236     P_sw[i] += e[i]; //add error
237     P_sw[i] = ((P_sw[i] % Q) + Q) % Q; //apply modulo Q
238 }
239 printf("Done.\n\n");
240
241 printf("Results (B)\n"); //print the results
242 print_array("Result p (Hardware)", P_hw, N);
243 printf("Cycles = %lu\n", (unsigned long)total_cycles_keygen_hw);
244 print_array("Result p (Software)", P_sw, N);
245 printf("Cycles = %lu\n", (unsigned long)total_cycles_keygen_sw);
246
247
248 //now, that we have public key, we can do encryption
249 const int16_t *pk_a = A;
250 // here, the SW result as the public key P
251 int16_t pk_p[N];
252 for(i=0; i<N; i++) pk_p[i] = (int16_t)P_sw[i]; //my new array pk_p (say at
    ↪ encryption time) is the public key
253
254 int32_t u_hw[N], v_hw[N]; //output u and v for hardware
255 int32_t u_sw[N], v_sw[N]; //output u and v for software
256
257 //test data, random
258 int16_t msg[N] = {0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1, 0, 1};
259 int16_t r[N] = {1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0};
260 int16_t e1[N] = {-1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1, 0, 1, -1};
261 int16_t e2[N] = {0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0};
262
263 printf("\nEncryption Step\n");
264 printf("Error e1: = ["); for(i=0; i<N; i++) printf("%d%s", e1[i], (i<N-1)?",
    ↪ " : ""); printf("]\n");
265 printf("Error e2: = ["); for(i=0; i<N; i++) printf("%d%s", e2[i], (i<N-1)?",
    ↪ " : ""); printf("]\n");
266 printf("Random r: = ["); for(i=0; i<N; i++) printf("%d%s", r[i], (i<N-1)?",
    ↪ " : ""); printf("]\n");
267 printf("Message m: = ["); for(i=0; i<N; i++) printf("%d%s", msg[i], (i<N-1)?",
    ↪ " : ""); printf("]\n");
268 printf("\n");
269 total_cycles_encrypt_hw = timed_ring_encrypt_hw(pk_a, pk_p, msg, r, e1, e2,
    ↪ u_hw, v_hw); //use accelerator
270 total_cycles_encrypt_sw = timed_ring_encrypt_sw(pk_a, pk_p, msg, r, e1, e2,
    ↪ u_sw, v_sw); //use software
271
272 printf("Results: \n"); //print the results
273 print_array("Ciphertext u (HW)", u_hw, N);

```

```
274     print_array("Ciphertext v (HW)", v_hw, N);
275     printf("Total HW cycles = %lu\n\n", (unsigned long)total_cycles_encrypt_hw);
276
277     print_array("Ciphertext u (SW)", u_sw, N);
278     print_array("Ciphertext v (SW)", v_sw, N);
279     printf("Total SW cycles = %lu\n\n", (unsigned long)total_cycles_encrypt_sw);
280
281     //now decryption part
282     int32_t m_dec_hw[N]; //decrypted message using hardware
283     int32_t m_dec_sw[N]; //decrypted message using software
284
285     printf("\nDecryption Step\n");
286     printf("Using Hardware...\n");
287     total_cycles_decrypt_hw = timed_ring_decrypt_hw(S, u_hw, v_hw, m_dec_hw);
288     ↪ //use accelerator
289     printf("Done.\n");
290
291     printf("Using Software...\n");
292     total_cycles_decrypt_sw = timed_ring_decrypt_sw(S, u_sw, v_sw, m_dec_sw);
293     ↪ //use software
294     printf("Done.\n\n");
295
296     printf("Decryption Results:\n");
297     print_array("Decrypted m (HW)", m_dec_hw, N);
298     printf("Cycles = %lu\n", (unsigned long)total_cycles_decrypt_hw);
299     print_array("Decrypted m (SW)", m_dec_sw, N);
300     printf("Cycles = %lu\n\n", (unsigned long)total_cycles_decrypt_sw);
301
302     //verify decryption - compare with original message
303     int hw_match = 1, sw_match = 1;
304     for(i=0; i<N; i++) {
305         if(m_dec_hw[i] != msg[i]) hw_match = 0;
306         if(m_dec_sw[i] != msg[i]) sw_match = 0;
307     }
308
309     if(hw_match) printf("SUCCESS: HW decryption matches original message!\n");
310     else printf("ERROR: HW decryption does NOT match original message.\n");
311
312     if(sw_match) printf("SUCCESS: SW decryption matches original message!\n");
313     else printf("ERROR: SW decryption does NOT match original message.\n");
314
315     printf("\nDone\n");
316
317     return 0;
318 }
```